



CHAPTER 8

STATIC AND DYNAMIC MEMORY ALLOCATION

Overview

- ▶ Memory Management
 - ▶ Memory
 - ▶ Memory Binding
- ▶ Memory Allocation
 - ▶ Static
 - ▶ Stack Based
 - ▶ Dynamic (Heap)

Memory Allocation

STATIC AND DYNAMIC
MEMORY ALLOCATION

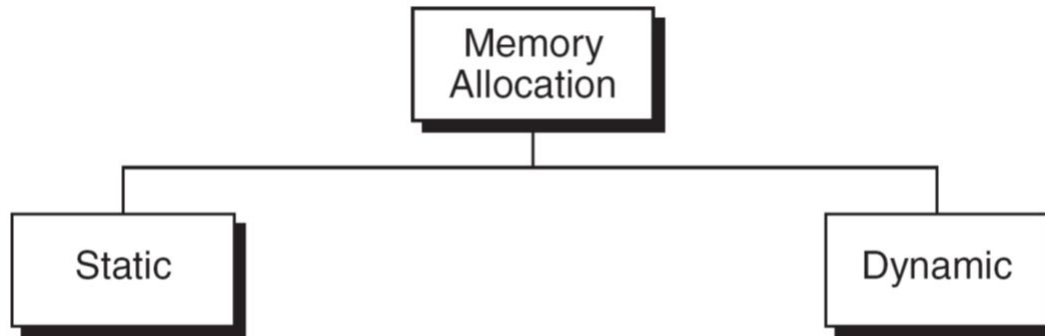
Memory

- ▶ Computer memory (RAM) is similar to a piece of coarse graph paper, with those boxes on it.
- ▶ Computer memory is one huge sheet of such paper, with each box representing one byte in memory.
- ▶ A byte can hold a number from 0 to 255. Each box on the paper has an address.
- ▶ The first one in the upper left has the number one, the second one just to its right has the number two and the last one on that line may have the number 1000 and so on...

Memory

- ▶ If we declare a variable named 'var', the computer picks the first empty box on the paper and makes a note that whenever we are saying 'var' we are actually referring to the contents of box on which it was written.
- ▶ The computers keep track of the first unused byte in memory by simply remembering it's address
- ▶ It uses stack pointer which points at the end of our used memory .

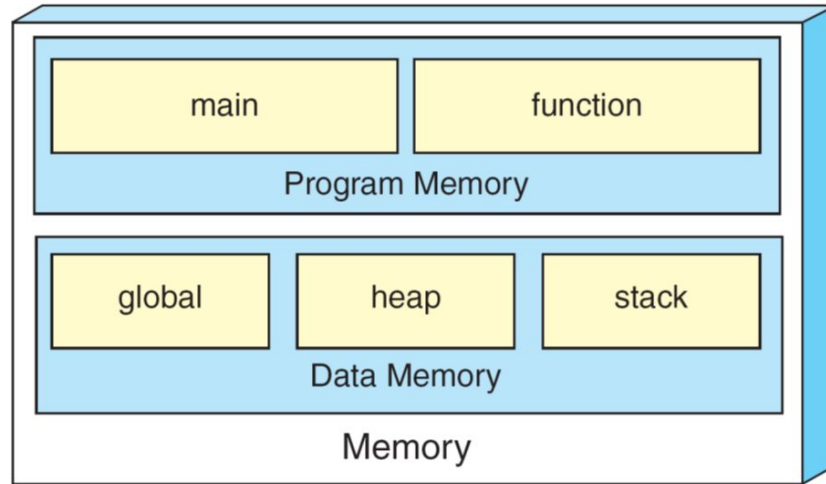
Memory Allocation



Memory Allocation

- ▶ Memory management refers to the process of managing the memory.
- ▶ It means allocation and deallocation of variables to the memory.
- ▶ Memory management is one of the crucial module in the Operating system.
- ▶ Memory can be allocated to the OS and user programs.
- ▶ Memory can be allocated both at compile time and during runtime

Conceptual View of Memory



Allocation on the Stack

- ▶ Stack
 - ▶ Easy to allocate (decrement esp)
 - ▶ Easy to deallocate (increment esp)
 - ▶ Automatic allocation at run-time, including variable size (alloca)
 - ▶ Can pass values to called procedures, but not up to callers

Allocation of Global Variables

- ▶ Global variables
 - ▶ Statically allocated
 - ▶ Have to decide at compile time how much space you need
 - ▶ Can pass values between any procedures

Allocation on the Heap

- ▶ Allocation on the heap
 - ▶ Dynamically allocated at run-time
 - ▶ Independent of procedure calls
 - ▶ But must be carefully managed
 - ▶ Automatically: garbage collection
 - ▶ Manually: malloc/free or new/delete

Problem Arising in Memory Allocation

- ▶ Let us consider we have a function that has three variables, two of them (A and C) are one byte in size each and the other (B) is variable in size.
- ▶ If the dynamic size variable is declared second it is stored in the middle of the other two variables.
- ▶ Initially all of the variables are one byte in size.

Problem Arising in Memory Allocation

1 2 3 4 5 6

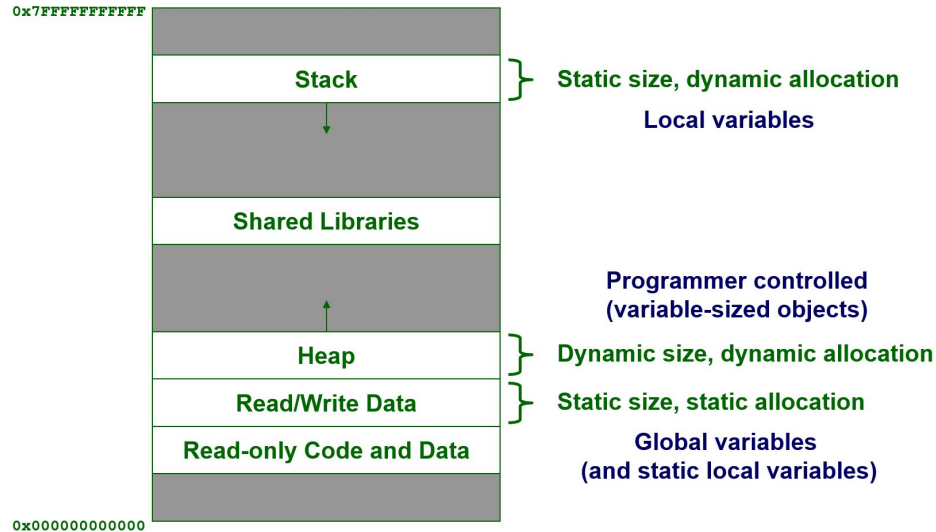
A B C

Now if B is enlarged to four bytes then

1 2 3 4 5 6

A B B B B C

Allocation in Process Memory



Problem Arising in Memory Allocation

- ▶ However, the code of the program does not remember the variables by name it remembers only the address of the variable.
- ▶ We moved the variable C from address 3 to 6.
- ▶ The next time the code tries to change the variable C, it will still write to the address 3, i.e., it overwrites the second byte of B.
- ▶ Therefore to overcome this problem the variable needed to have a fixed size and must be known at compile time.

Binding and Binding times

- ▶ A memory binding is an association between 'memory address' attributes of data item and the address of a memory area.
- ▶ Memory allocation is the procedure used to perform memory binding and the ***binding time*** is the time taken to perform the binding.
- ▶ The memory allocation can be generally static and dynamical in nature.

Static Memory Allocation

Static Memory Allocation

- ▶ Memory binding for *Static Memory Allocation* is typically performed during the compilation of a program.
- ▶ No memory allocation and deallocation is performed during the execution of the program.
- ▶ Therefore, the variables are permanently allocated.

Static Memory Allocation

- ▶ It corresponds to file scope variables and local static variables.
- ▶ The following type of variables are statically allocated:
 - ▶ All global variables, regardless of whether or not they have been declared static.
 - ▶ Local variables explicitly declared to be static

Properties of Statically Allocated Variables

- ▶ Statically allocated variables have their storage allocated and initialized before main starts running.
- ▶ Statically allocated local variables are not reinitialized on every call to the function in which they are declared.
- ▶ The address and the size of these allocations are fixed at the time of compilation.

Stack Memory Allocation

- ▶ The second type of memory allocation is the stack memory allocation, which corresponds to the non static variables and call by value parameter variables.
- ▶ The size of these allocations are fixed at the time of compilation but their address will vary depending on when the function which defines them is called.
- ▶ This memory allocation is placed in a system memory area called stack.
- ▶ When a function is called , the state of the calling function must be preserved so that when the called function returns the function can resume the execution.

Java Stack

- ▶ A Java stack is part of the computer's memory where temporary variables, which are created by all functions that are used, are stored.
- ▶ It is used to execute a thread and may have certain short-lived values as well as references to other objects.
- ▶ It uses a LIFO data structure, or last in first out.

Java Stack Usage

1. When a method is invoked, it creates a new block in the stack for that particular method.
2. The new block will have all the local values, as well as references to other objects that are being used by the method.
3. When the method ends, the new block will be erased and will be available for use by the next method.

Java Stack Usage

- ▶ The objects you find here are only accessible to that particular function and will not live beyond it.
- ▶ This makes it very easy to keep track of the stack, where the latest reserved block is also the first to be freed.
- ▶ The variables created for the method are directly stored in the memory, allowing for fast access.
- ▶ The memory size of a Java stack is generally much less than in a Java heap space because when a method ends, all the variables created on the stack are erased forever.

Dynamic Memory Allocation

STATIC AND DYNAMIC
MEMORY ALLOCATION

Dynamic Memory Allocation

- ▶ This type of binding is performed during the execution of the program.
- ▶ So the memory bindings are established and destroyed during the execution of the program.

Example

- ▶ The fig of next slide illustrates the static and dynamic binding to a program consisting three program units (A, B and C)

1. Shows the static memory allocation
2. Shows the dynamic memory allocation when only program unit A is active.
3. Shows the situation where A calls B.
4. The last part shows the situations after B returns to A and A calls C.



Dynamic Memory Allocation

- ▶ These memory allocation are called dynamic because their location and size can vary throughout the lifetime of the program.
- ▶ Programmatically, memory allocation is via `malloc()` (C language) or `new` operator (Java language).
- ▶ The size, address and contents of such memory vary at runtime and so can cause a lot of problem when trying to diagnose a fault in a program.
- ▶ These memory allocations are placed in a system memory area called heap, which are allocated per process on some system.

Application
Dynamic Memory Allocator
Heap Memory

Memory Allocation

- ▶ Explicit vs. Implicit Memory Allocator
 - ▶ Explicit: application allocates and frees space
 - ▶ E.g., malloc and free in C, new/delete/delete[] in C++
 - ▶ Implicit: application allocates, but does not free space
 - ▶ E.g. garbage collection in Java, ML or Lisp
- ▶ Allocation
 - ▶ In both cases the memory allocator provides an abstraction of memory as a set of blocks
 - ▶ Provides free memory blocks to application
- ▶ Allocator is typically a system or language library

Java Heap

- ▶ Java objects are in an area, which is called the heap.
- ▶ It is created when the program is run, and its size may decrease or increase as your program runs.
- ▶ It can easily become full, and when it does, garbage collection is initiated.
- ▶ This is when objects that are no longer used are deleted to make way for new objects.

Java Heap

- ▶ Unlike in a Java stack where memory allocation is done when your program is compiled, in a heap it is allocated as your program is run.
- ▶ Accessing variables placed here is a bit slower compared to a stack's direct and fast access.
- ▶ Heap is likened to a *global memory pool*.
- ▶ A method or function will use the heap for memory allocation if you need the data or variables to live longer than the method or function in question.
- ▶ The objects you find here are accessible to all the functions.

Java Heap

- ▶ Also, there is no specific order in reserving blocks in a heap.
- ▶ You can allocate blocks at any time, and then you can free it when you wish
- ▶ It is much more complex to keep track of the parts that are free and can be allocated, but it can also be divided into two generations or sub-areas.
- ▶ These sub-areas are called the young space (or nursery) and the old space.

Java Heap

- ▶ The young space is typically reserved for the memory allocation for new objects.
- ▶ When the young space becomes full, *garbage collection* happens.
- ▶ Short-lived or temporary objects typically use the young space.
- ▶ This helps make garbage collection faster when compared to a heap without any divisions.

Stack vs. Heap

Stack	Heap
The size of the stack will vary as methods and functions create and delete local variables as needed.	Memory is not managed automatically nor is it as tightly managed by the central processing unit the way stack is managed. You would need to free allocated memory yourself when these blocks are no longer needed.
Memory is allocated and then subsequently freed without you needing to manage the memory allocation.	The heap is prone to memory leaks, where memory is allocated to unused objects and will not be available to processes other than that.
Stack has size limits, which can vary according to the operating system you use	There is no size limit in the heap.

Static vs. Dynamic Memory Allocation

Static Memory Allocation	Dynamic Memory Allocation
variables get allocated permanently	variables get allocated only if a program unit gets active
memory allocation is done before program execution	memory allocation is done during program execution
uses the data structure called stack for implementing static allocation	uses the data structure called heap for implementing dynamic allocation
less efficient	more efficient
there is no memory reusability	there is memory reusability and memory can be freed when not required

END OF CHAPTER 8

STATIC AND DYNAMIC MEMORY ALLOCATION